



La base fuera de la máquina (Supabase)

Cómo sacar PostgreSQL del VPS y ponerlo en Supabase,
qué cuesta y por qué ****no es el camino recomendado****.
Probado con una base externa de verdad; contra Supabase
mismo, no.

La recomendación, primero

PostgreSQL en el propio VPS, que es lo que hace `compose.prod.yml` sin tocar nada.

	En el VPS (por defecto)	En Supabase
Piezas que pueden fallar	Una menos	La red entre las dos, y un proveedor más
Se pausa sola	No	Sí: el plan gratis, a los 7 días sin actividad
Copias	<code>scripts/backup.sh</code> , ya funciona	Hay que cambiarlo (abajo)
Recursos	12 GB de RAM para una base que empieza en 73 MB	Los del plan
Contrato de encargado (art. 28.4)	Solo Contabo	Contabo y Supabase

Con 12 GB en el VPS, mover la base fuera añade piezas y no quita ninguna. Este documento existe para que la opción esté **documentada y probada**, no para recomendarla.

Cuándo sí tendría sentido: si no quieres administrar Postgres en absoluto, si quieres su consola web para mirar datos, o si algún día hace falta réplica gestionada —que ya es plan de pago—.

1. La cadena de conexión: usa el *Session pooler*

En el panel de Supabase, **Project Settings** → **Database** → **Connection string**, y de ahí la del **Session pooler**.

No uses la conexión directa (`db.<ref>.supabase.co`): **resuelve solo por IPv6**, y muchos servidores —incluidos VPS baratos— no tienen IPv6 de salida. El síntoma es un `ENETUNREACH` o un tiempo de espera agotado al arrancar, que parece un problema de credenciales y no lo es. El pooler responde por IPv4.

```
DATABASE_URL=postgresql://postgres.<ref>:<CONTRASEÑA>@aws-0-<región>.pooler.supabase.com:5432
```

sslmode: pon `verify-full`, y aquí está el porqué

Comprobado sobre la versión que trae este repositorio (`pg-connection-string` 2.14): `sslmode=require` hoy se trata como `verify-full`, es decir, verifica de verdad el certificado. Pero la propia biblioteca avisa de que en su próxima versión mayor pasará a la semántica de libpq, donde `require` cifra pero no verifica nada.

O sea: escribir `require` significa hoy una cosa y mañana la contraria, sin que tú cambies nada. **Escribe `verify-full`**, que significa lo mismo en las dos y es lo que quieres: si alguien se pone en medio, la conexión falla en vez de seguir.

Si al arrancar ves un error de certificado, es esto y no las credenciales.

2. Levantar sin el contenedor de base

Hay un archivo para esto, `compose.supabase.yml`, y se añade al de siempre:

```
docker compose -f compose.prod.yml -f compose.supabase.yml --env-file .env up -d
```

Hace tres cosas, y las tres hacen falta:

- deja el servicio `db` en un perfil que nunca se activa, así que no se levanta;
- borra la espera por `db` en `migrar`, que si no dejaría el `up` colgado para siempre;
- y en `api` **reemplaza la lista de dependencias entera** en vez de borrarla, para que siga esperando a que `migrar` termine.

Ese último punto costó descubrirlo y merece una frase: `depends_on` **se fusiona por clave**, así que añadir no quita la espera por `db`; pero borrarlo entero se lleva también la espera por `migrar`, y entonces la API empieza a atender con la base a medio migrar. Se comprobó: con el borrado entero, `migrar` y `api` arrancaban **a la vez**. La solución está comentada dentro del archivo.

Una rareza que verás

`POSTGRES_PASSWORD` **sigue haciendo falta en el `.env`** aunque no se use para nada: compose interpola el archivo **entero** antes de decidir qué servicios levanta, así que la variable del servicio `db` se sigue leyendo aunque `db` no arranque. Déjala con cualquier valor. Si la borras, el `up` falla con «required variable POSTGRES_PASSWORD is missing a value».

Comprobar

```
docker compose -f compose.prod.yml -f compose.supabase.yml --env-file .env ps -a
```

`api` en `healthy`, `caddy` en `running`, `migrar` en `Exited (0)` y **ningún `db`**.

3. Los límites del plan gratis

- **Se pausa a los 7 días sin actividad.** Y este producto tiene, por diseño, semanas sin actividad: se manda un pase, se abre, y no vuelve a pasar nada. Un cliente que abra un enlace con el proyecto pausado se encuentra la aplicación caída. Lo bueno, por cómo está escrito el consumo: **el pase no se gasta**, porque consumirlo es un `UPDATE` que nunca llega a ejecutarse. Sigue abriéndose cuando el proyecto despierte. (Esto es lectura del código, no una prueba: no se ha provocado una pausa de verdad.)
- **No incluye copias de seguridad.** Ninguna. Lee el apartado siguiente antes de decidir.
- Hay topes de tamaño y de conexiones simultáneas según el plan; el pooler es justo lo que evita quedarse sin conexiones.

4. Las copias: esto es lo que puede costarte caro

`scripts/backup.sh` NO funciona con esta variante, y falla del todo, no a medias. Comprobado:

```
Volcando vista a ./copias/vista-20260903T102836Z.dump
service "db" is not running
```

El script vuelca con `docker compose exec -T db pg_dump`, y aquí no hay servicio `db`. Súmalo a que el plan gratis **no hace copias**, y el resultado de elegir Supabase sin tocar nada más es **quedarse sin ninguna copia de seguridad**, creyendo que las hay.

El sustituto, **probado contra una base externa de verdad**, es un `pg_dump` desde un contenedor de una sola vez:

```
docker run --rm -v "$PWD/copias:/copias" --env-file .env postgres:16-alpine \
  sh -c 'pg_dump "$DATABASE_URL" -Fc -f /copias/vista-$(date -u +%Y%m%dT%H%M%S).dump'
```

Y comprobar que el volcado se puede leer, que es la mitad del trabajo —un archivo corrupto también existe —:

```
docker run --rm -v "$PWD/copias:/copias" postgres:16-alpine \
  pg_restore -l /copias/<el-archivo>.dump | head -3
```

Dos avisos:

- `pg_dump` **tiene que ser de una versión igual o mayor que el servidor.** Si Supabase corre una versión más nueva que la imagen que uses, el volcado falla con un mensaje sobre versiones. Cambia `postgres:16-alpine` por la que toque.
- Lo de arriba **no rota nada ni comprueba nada más.** `backup.sh` sí hace las dos cosas, y en el orden correcto (comprobar la copia nueva **antes** de borrar las viejas, para que una noche con la base caída no se lleve las buenas).

Si acabas eligiendo Supabase, hay que adaptar `backup.sh`, no basta con pegar el comando de arriba en el cron. No se ha hecho porque el camino recomendado es el otro y tocar el script pondría en riesgo lo que hoy funciona.

5. Volver atrás

Se puede, y conviene saberlo antes de irse. Vuelca desde Supabase con el comando del apartado 4, quita el `-f compose.supabase.yml`, levanta con el `db` local y restaura:

```
docker compose -f compose.prod.yml --env-file .env up -d db
docker compose -f compose.prod.yml exec -T db \
    pg_restore -U vistta -d vistta --clean --if-exists /copias/<el-archivo>.dump
```

No ejecutado: la restauración *desde Supabase* no se ha probado. La restauración local sí está descrita y probada en `DESPLIEGUE.md`.

Qué se ha ensayado y qué no

Probado de verdad	La variante entera contra una PostgreSQL externa al despliegue: <code>compose.supabase.yml</code> , migraciones aplicadas fuera, <code>api</code> sana sin contenedor <code>db</code> , alta de cuenta, y el pase abriendo 200 y luego 410
Probado también	Que <code>backup.sh</code> falla con <code>service "db" is not running</code> , y que el <code>pg_dump</code> sustituto produce un volcado que <code>pg_restore -l</code> lee
Comprobado leyendo	Que <code>sslmode=require</code> hoy equivale a <code>verify-full</code> en la versión que trae el repositorio, y que la biblioteca avisa de que eso va a cambiar
Nunca ejecutado	Supabase: ni el pooler, ni el IPv6 de la conexión directa, ni el TLS contra sus certificados, ni la pausa a los 7 días

Lo que se ha probado es la **forma** del despliegue con la base fuera, que es donde estaban las sorpresas. Lo que queda por ver es el proveedor.